INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Licenciatura em Engenharia Informática

SPRINT 3

Documentação Das Alterações Realizadas Pelo Grupo G092

Iva Silva, 1241176

João Ferreira, 1241166

José Cardoso, 1241323

Rafael Capão, 1241285

G092

Prof. Helena Brás

Prof. Alexandra Gavina

Prof. Nuno Escudeiro

Prof. Catarina de Oliveira

Prof. Jorge Duarte

Index

Discrete Mathematics 3

US14 3

01. More Accurate Station Initialization 3

02. Improved Graph Visualization for Disconnected Nodes 4

03. Redefined Error Messaging for Disconnected Graphs 4

Figure Index

Figure 1 3

Figure 2 4

Figure 3: Snippet example of railway.dot file 5

Discrete Mathematics

After defending the work developed during Sprint 2, the group took into great consideration the recommendations given by Professor Alexandra Gavina, and worked to improve the work already done, as well as maintain the type of development for the intended work in Sprint 3.

The changes made by the group aim to improve the user experience, expand analytical capabilities, and provide deeper insights into station operational performance.

US14

01. More Accurate Station Initialization

The previous code worked in a way where the graph was built only by adding nodes that appeared in the filtered edge list. With the added improvement, it collects all stations from the CSV file, ensuring that even isolated bulges—bulges with no connecting edges after filtering—are added to the graph.

```
Set<String> allStations = new HashSet<>();
for (String[] edge : edges) {
    allStations.add(edge[0]);
    allStations.add(edge[1]);
}
Map<String, List<String>> graph = new HashMap<>();
for (String station : allStations) {
    graph.put(station, new ArrayList<>());
}
for (String[] edge : edges) {
    graph.get(edge[0]).add(edge[1]);
    graph.get(edge[1]).add(edge[0]);
}
```

Figure 1

This change ensures that stations that aren't included in any route (i.e., in electrified-only mode) are still present in the graph structure, which supports better visualization and complete reporting.

02. Improved Graph Visualization for Disconnected Nodes

In the improved code, when generating the file for the railway graph, the program explicitly adds isolated stations to the .dot file so they appear in the visual output.

```
List<String> odd = new ArrayList<>();
for (String node : graph.keySet()) {
    if (graph.get(node).size() % 2 != 0) {
        odd.add(node);
    }
}
```

Figure 2

This ensures all stations are shown in the resulting .svg file, not just those connected by routes. It improves the completeness and accuracy of the visual representation, especially important in maintenance planning.

03. Redefined Error Messaging for Disconnected Graphs

The message shown when the graph is disconnected has been made more specific and clearer.

In the previous coding method, the output message was: “The network is disconnected. No maintenance route is possible.”

However, in the improved one, the output message is: “no maintenance route is possible due to disconnection.”

This change, while minor, provides a clear, less technical explanation, improving the user interface when utilising the application.

```
graph G {
  "S_Kyle_of_Lochalsh" -- "D_Attadale" [color=red];
  "D_Attadale" -- "S_Dingwall" [color=red];
  "S_Dingwall" -- "S_Ardgay" [color=red];
  "S_Ardgay" -- "D_Helmsdale" [color=red];
  "D_Helmsdale" -- "S_Wick" [color=red];
  "S_Dingwall" -- "T_Inverness" [color=red];
  "T_Inverness" -- "S_Elgin" [color=yellow];
  "S_Elgin" -- "T_Aberdeen" [color=yellow];
  "T_Aberdeen" -- "S_Arbroath" [color=yellow];
  "S_Arbroath" -- "T_Dundee" [color=yellow];
  "T_Dundee" -- "D_Inverkeithing" [color=red];
  "D_Inverkeithing" -- "T_Edinburgh" [color=red];
  "T_Inverness" -- "D_Aviemore" [color=red];
  "D_Aviemore" -- "S_Kingussie" [color=red];
```

Figure 3: Snippet example of railway.dot file